The following listing shows sequential consistency in action. The loads and stores to `x` and `y` are explicitly tagged with `memory_order_seq_cst`, although this tag could be omitted in this case because it's the default.

Listing 5.4. Sequential consistency implies a total ordering

```
#include <atomic>
#include <thread>
#include <assert.h>

std::atomic<bool> x,y;
std::atomic<int> z;

void write_x()
{
    x.store(true,std::memory_order_seq_cst);    <-(1)
}

void write_y()
{
    y.store(true,std::memory_order_seq_cst);    <-(2)
}

void read_x_then_y()
{
    while(!x.load(std::memory_order_seq_cst));
    if(y.load(std::memory_order_seq_cst))       <-(3)
        ++z;
}

void read_y_then_x()
{
    while(!y.load(std::memory_order_seq_cst));
    if(x.load(std::memory_order_seq_cst))       <-(4)
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x);
    std::thread b(write_y);
    std::thread c(read_x_then_y);
    std::thread d(read_y_then_x);
    a.join();
    b.join();
    c.join();
    d.join();
    assert(z.load()!=0);    <-(5)
}
```

The `assert` (5) can never fire, because either the store to `x` (1) or the store to `y` (2) must happen first, even though it's not specified which. If the load of `y` in `read_x_then_y` (3) returns `false`, the store to `x` must occur before the store to `y`, in which case the load of `x` in `read_y_then_x` (4) must return `true`, because the `while` loop ensures that the `y` is `true` at